

ASA: Ant Social Architecture — A Compilable Governance Framework for Multi-Agent Cognitive Operating Systems

Suode ()

Independent Researcher

`suode.works@proton.me`

<https://github.com/suodeworks>

Abstract

This paper describes the Ant Social Architecture (ASA), a framework developed over approximately two years (2024–2026) that approaches multi-agent AI coordination as a compilation and governance problem rather than an orchestration problem. The central observation motivating this work is that current multi-agent systems ask language models to do too many things at once: parse human intent, extract constraints, find evidence, plan execution, and verify their own output—all in a single pass. ASA addresses this by separating concerns through a semantic compilation pipeline that converts natural language into typed protocol objects (HPS, ISG, CRM, EBP, MIR, ECP, MHE) before execution begins. These protocols are governed by a compilable constitution ("mother law") and enforced through contractual node roles with explicit escalation and rollback rules. The architecture was developed iteratively across six versions: V1 laid the protocol and compilation backbone; V2 added runtime execution loops, teacher distillation, and tool integration; V3 introduced mechanisms for long-term system evolution; V4 addressed self-governance boundaries; V5 extended the model to federated multi-system networks with sovereignty constraints; V6 added resource economics and precedent-based arbitration. This paper presents the complete specification, compares it against existing frameworks, and outlines an implementation path. The work represents an independent effort that, as we discuss in Section 2, converges with several research directions that have since appeared in the published literature.

Keywords: Multi-Agent Systems, Cognitive Operating Systems, Semantic Compilation, Constitutional AI Governance, Federated Agent Architecture, Protocol-Driven Execution

1 Introduction

Anyone who has spent time building with large language models knows the frustration: you give a model a complex task, and it spends most of its effort just figuring out what

you meant rather than doing the actual work. This is not a capability problem—the models are plenty capable. It is an interface problem. When a model receives a raw prompt, it has to simultaneously understand intent, figure out constraints, recall relevant knowledge, plan a sequence of actions, execute them, and check its own work. That is too many jobs for one inference pass, and the result is that most of the model’s computational budget goes to cognitive overhead rather than high-value reasoning.

Existing multi-agent frameworks like AutoGen, CrewAI, and LangGraph partially address this by distributing work across multiple agents. But they share a common limitation: agents still communicate in natural language, operate without formal contracts, and have no systematic way to ensure that the original human intent survives intact through a chain of handoffs. What gets lost is not information in the narrow sense—it is structure: the constraints, the priorities, the boundary conditions, the things you cannot do.

ASA (Ant Social Architecture) grew out of an attempt to solve this problem from first principles. The core idea is simple: treat natural language as source code, not as execution language. Just as a C compiler transforms human-readable code into machine instructions before execution, ASA interposes a semantic compilation layer that transforms prompts into typed, validated protocol objects before any agent acts on them. This compilation is governed by a formal constitution—a "mother law" document that is itself compilable into enforceable rules, role assignments, and runtime constraint packages.

The architecture was not designed all at once. It grew over roughly two years of iterative development, from an initial protocol backbone (V1, 2024) through runtime mechanics (V2), evolution mechanisms (V3), self-governance (V4), federation across independent systems (V5), to resource economics and legal precedent (V6, completed April 2026). Each version addressed problems that became visible only after the previous version was complete—the kind of design evolution that is difficult to plan in advance and easier to narrate in retrospect.

This paper presents the full specification, places it in context with recent work that has appeared in parallel, and describes a concrete implementation path.

2 Related Work

2.1 Multi-Agent LLM Frameworks

Several multi-agent frameworks appeared between 2023 and 2025. AutoGen [13] lets agents converse flexibly but leaves inter-agent communication in natural language—no typed protocol guarantees. CrewAI [10] adds role-based delegation; LangGraph [13] provides stateful graph workflows. MetaGPT [5] is perhaps the most interesting of the group: it encodes Standard Operating Procedures into collaboration, getting closer to formal protocol compilation without quite arriving there. AgentScope [3] offers customizable message-passing.

DSPy [21] is the closest relative in spirit—it compiles declarative LLM calls into optimized pipelines, sharing the insight that model interactions benefit from compilation. But DSPy compiles for prompt optimization (making the same call work better), while ASA compiles for governance (transforming intent into typed protocol objects that carry constraints, evidence, and audit trails through the execution chain). CoALA [22] organizes agents through cognitive architecture principles with memory modules and decision procedures, though it stops short of formalizing inter-agent protocols.

The gap that remained open across all these systems: none of them treat the human prompt as source code that must be compiled before execution, and none enforce a constitutional governance layer over agent interactions.

2.2 Agent Operating Systems and Inter-Agent Protocols

The "OS for agents" idea has been explored from several angles. AIOS [9] manages agent scheduling, context, and tool integration at the OS level. OS-Copilot [13] targets generalist computer interaction. AgentOS (Chen et al., 2024) adds hierarchical agent management with resource allocation.

At the protocol level, Anthropic’s Model Context Protocol (MCP, 2024) standardizes how models connect to external tools through a client-server architecture—solving tool integration but not inter-agent governance. Google’s Agent-to-Agent (A2A) Protocol (2025) standardizes communication between heterogeneous agents, focusing on interoperability.

These systems handle resource management, scheduling, and interoperability. What they leave untouched: constitutional governance, semantic compilation from natural language to machine protocols, adversarial verification chains. ASA’s "operating system" metaphor extends further—it includes legislative (mother law), judicial (arbitration), and executive (node society) branches. This choice has a cost: the system is heavier to bootstrap than a simple orchestration framework, and the governance overhead only pays off at a certain complexity threshold. For simple two-agent workflows, AutoGen or CrewAI remain more practical.

2.3 AI Governance and Constitutional Approaches

Constitutional AI [1] showed that constitutional principles can guide model behavior through self-critique and revision. Collective Constitutional AI [18] extended this to democratically-sourced constitutions. Democratic AI [6] explored mechanism design for governance.

In multi-agent settings: Governing AI Agents [2] proposes policy frameworks for autonomous agent systems. Practices for Governing Agentic AI Systems [12] discusses regulatory approaches. Multi-Agent Risks from Advanced AI [4] maps the risk landscape.

ASA takes a different angle. Rather than treating constitutional principles as behavioral guidelines that a model internalizes, ASA makes the constitution a compilable source document—parsed into clause registries, compiled into runtime constitution packs, enforced through verification gates. The constitution is not advice; it is code. This was not the original design intent—early versions (2024) used the mother law as a long system prompt. The compilation approach emerged from a practical observation: long constitutional prompts degrade as context windows fill up, while compiled clause objects remain stable regardless of context length.

2.4 Federated and Distributed Agent Systems

Federated Learning [8] established principles for distributed training with privacy preservation. AgentNet [15] proposes decentralized evolutionary coordination for LLM-based

multi-agent systems. Decentralized Governance of AI Agents [7] addresses governance frameworks for inter-system coordination.

Multi-Agent Societies [11, 3] simulate social dynamics among LLM agents but lack formal sovereignty boundaries or economic equilibrium constraints. ASA’s V5-V6 federation model (completed early 2026) introduces sovereignty profiles, treaty contracts, trust attestation, and civilizational economics—concepts that may be over-engineered for current deployment scales. They were designed for a future where multiple independently-governed agent systems need to interoperate, a scenario that is approaching but has not yet arrived at production scale.

2.5 Formal Verification in Agent Systems

Dennis et al. (2024) apply model checking to multi-agent programs. A Trace-Based Assurance Framework for Agentic AI Orchestration [17] proposes execution-trace-based verification for agent behavior compliance. Guardrails AI (2024) provides practical output validation.

ASA’s verification approach combines structural checking (protocol schema compliance), semantic checking (logical consistency), evidence checking (claim-evidence alignment), and adversarial counterexample generation. The counterexample chain—where a dedicated node actively tries to break the output before delivery—was the last piece added to V1, after observing that verifier-only systems tend to confirm rather than challenge. There is an unresolved tension here: aggressive counterexample generation increases quality but also increases latency and cost. The route depth policy modulates counterexample intensity, but the optimal balance remains empirical and likely task-dependent.

3 System Architecture Overview

3.1 Foundational Thesis

The single thesis underlying ASA: natural language is not an execution language; it is task source code. When models consume raw natural language, they must simultaneously perform intent extraction, constraint identification, evidence gathering, execution planning, and self-verification—too many cognitive functions competing for the same inference budget. ASA eliminates this by interposing a semantic compilation layer that produces typed, validated, machine-readable protocol objects for downstream consumption.

3.2 Nine Architectural Planes

The system is organized into nine orthogonal planes:

These planes are vertically orthogonal: removing any one degrades the system from a governed execution civilization to a multi-model chat flow. The nine-plane structure crystallized around mid-2024 after several failed attempts—organizing by layer felt too hierarchical, organizing by module felt too flat. Planes turned out to be the right abstraction because they can intersect without creating dependency chains.

3.3 The Unique Main Chain

The entire system compresses to a single canonical execution chain:

Plane	Responsibility
Rule Plane	Mother law as compilable constitutional source; clause registries; role/task law slices
Compilation Plane	Semantic compiler: segmentation, intent decomposition, ambiguity detection, constrain
Protocol Plane	HPS/ISG/CRM/EBP/MIR/ECP/MHE as the typed data backplane
Execution Plane	Node society as contractual execution graph with I/O contracts
Scheduling Plane	Queen Router: value/risk/uncertainty/cost unified decision
Truth Plane	Supervision chain, counterexample chain, final audit, assertion layer
Runtime Plane	Six-step council, ten-phase lifecycle, node state machine, route depth policy
Engineering Plane	Package architecture: protocol-types, constitution, prompt-compiler, node-runtime, etc
Evolution Plane	Experiments, convergence criteria, log reflux, versioning, migration

Mother Law TXT

Anchor Index / Clause Registry / Clause Graph
 Role Slice / Task Overlay / Constitution Pack
 HPS (Human Prompt Snapshot)
 ISG (Intent-Structured Goal)
 CRM (Constraint-Risk Matrix)
 EBP (Evidence Base Package)
 MIR (Machine Intent Representation)
 ECP (Execution Contract Protocol)
 Route Decision
 Node Graph Execution
 MHE (Machine Handoff Envelope) Handoffs
 Verifier Counterexample Core Arbiter
 Final Output Auditor
 User Delivery
 Audit Log / Experiment Log / Migration Log

This chain encodes a strict ordering: the mother law is compiled first; user input is semantically compiled; high-value judgments are evidence-backed and constraint-bound; the core model performs only high-value arbitration; all outputs pass verification, counterexample challenge, and final audit. None of this was designed top-down. It emerged from repeatedly asking "what must be true before this step can execute correctly?" and working backward from delivery to input.

4 The Protocol Type System (V1 Canonical Schema)

4.1 Design Principles

An early design decision that proved critical: use a unified canonical schema rather than separating conceptual and runtime representations. All protocol objects share a common metadata base class:

```

1 type ProtocolName =
2   | "HPS" | "ISG" | "CRM" | "EBP"
3   | "MIR" | "ECP" | "MHE"
4   | "EVIDENCE" | "RISK" | "ROUTE_DECISION";
5

```

```

6 interface ProtocolMeta {
7   protocol: ProtocolName;
8   version: string;
9   objectId: string;
10  taskId: string;
11  traceId: string;
12  producedAt: string;
13  producer: string;
14  parentRefs?: string[];
15  sourceRefs?: string[];
16 }

```

This eliminates a problem that plagued early prototypes: some objects had IDs and some didn't; some were traceable and some relied on context guessing. The `ProtocolMeta` base class makes traceability non-optional.

4.2 HPS — Human Prompt Snapshot

The HPS captures raw human input with zero information loss. It preserves source messages, referenced files, session context, user locale, and user tone classification. The critical invariant is `lossless: true`—no inference, summarization, or interpretation contaminates the HPS. The initial temptation was to do light normalization at this stage (fixing typos, expanding abbreviations), but this was rejected because any transformation at the HPS level makes it impossible to later diagnose whether a downstream error originated from the user's actual words or from an overeager normalizer.

```

1 interface HPS {
2   meta: ProtocolMeta;
3   rawText: string;
4   sourceMessages: Array<{
5     messageId: string;
6     role: "user" | "system" | "developer" | "assistant";
7     text: string;
8     timestamp?: string;
9   }>;
10  referencedFiles: Array<{
11    path: string;
12    kind: "code" | "doc" | "config" | "asset" | "unknown";
13  }>;
14  sessionContextRefs?: string[];
15  userLocale?: string;
16  userTone?: "directive" | "exploratory" | "uncertain"
17           | "emotional" | "mixed";
18  lossless: true;
19 }

```

Scenario Example: A user submits: `exitit"Refactor the authentication module to use JWT tokens instead of session cookies, but keep backward compatibility for mobile clients still on v2.3."` The HPS captures this verbatim along with the referenced codebase files, the user's directive tone, and session context (prior discussion about mobile client constraints). No interpretation occurs—the raw intent, including the implicit tension be-

tween "refactor" and "backward compatibility," is preserved losslessly for downstream compilation stages.

4.3 ISG — Intent-Structured Goal

The ISG decomposes the raw prompt into a structured goal hierarchy with explicit dependency graphs, ambiguity items with severity classification, and assumption candidates. The design choice that matters most here: ambiguities are first-class objects, not footnotes. An unresolved high-severity ambiguity blocks downstream ECP issuance—the system refuses to proceed rather than guessing.

```
1 interface ISG {
2   meta: ProtocolMeta;
3   primaryGoal: string;
4   subGoals: GoalNode[]; // {id, label, priority, dependsOn[]}
5   taskTypeCandidate: "architecture" | "implementation" | "
      documentation"
6   | "review" | "audit" | "research" | "migration" | "hybrid";
7   completionCriteria: string[];
8   explicitConstraints: string[];
9   implicitConstraints: string[];
10  forbiddenActions: string[];
11  requestedArtifacts: string[];
12  ambiguityItems: AmbiguityItem[]; // {id, kind, description,
      severity, resolved}
13  assumptionCandidates: string[];
14 }
```

Scenario Example: From the JWT refactoring HPS, the ISG decomposes the intent into: primaryGoal: "Replace session-cookie auth with JWT", subGoals: [{label: "Implement JWT issuance", priority: 1}, {label: "Maintain v2.3 mobile backward compat", priority: 2, dependsOn: ["jwt-issuance"]}], It flags an ambiguity item: {kind: "scope", description: "Does backward compatibility mean dual-auth or translation layer?", severity: "high", resolved: false}. This unresolved ambiguity will block ECP issuance until clarified.

4.4 CRM — Constraint-Risk Matrix

The CRM separates hard constraints, soft constraints, system constraints, and confidentiality constraints into distinct categories. It also introduces a formal resource budget structure and typed risk flags. The four-way constraint separation was arrived at after discovering that mixing "must not break mobile sessions" (hard) with "prefer minimal token size" (soft) caused downstream nodes to treat all constraints with equal weight—or worse, to silently drop soft constraints when under budget pressure.

```
1 interface CRM {
2   meta: ProtocolMeta;
3   hardConstraints: string[];
4   softConstraints: string[];
5   systemConstraints: string[];
6   confidentialityConstraints: string[];
```

```

7   forbiddenActions: string[];
8   resourceBudget: {
9       budgetTier: "B0" | "B1" | "B2" | "B3";
10      latencyMode: "fast" | "standard" | "deep" | "audit";
11      maxEscalationDepth: number;
12      maxDelegations: number;
13      maxCoreCalls: number;
14  };
15  riskFlags: RiskFlag[]; // typed: ambiguity, hallucination,
16                          // leakage, cost_ouerrun,
17                          // protocol_drift, etc.
18  }

```

Scenario Example: For the JWT refactoring task, the CRM identifies: `hardConstraints`: ["Must not break existing v2.3 mobile sessions"], `softConstraints`: ["Prefer minimal token payload size"], `systemConstraints`: ["Auth module must remain stateless"], `resourceBudget`: {`budgetTier`: "B2", `latencyMode`: "standard", `maxCoreCalls`: 3}, and `riskFlags`: [{`type`: "compatibility", `severity`: "high", `reason`: "Dual-auth period may introduce session hijacking vectors"}].

4.5 EBP — Evidence Base Package

The EBP collects evidence items with source attribution, confidence scores, relevance mapping, and conflict detection. The most important field is `missingEvidence`—the system explicitly tracks what it does not know, rather than proceeding on incomplete information and hoping for the best.

```

1  interface EBP {
2      meta: ProtocolMeta;
3      evidenceItems: EvidenceItem[]; // {id, sourceType, sourceRef,
4                                     // claim, confidence,
5                                     // relevanceTo[]}
6      missingEvidence: string[];
7      conflictPairs: Array<{left: string; right: string; reason:
8                             string}>;
9      coverageSummary?: string;
10  }

```

Scenario Example: The evidence scout node gathers: `evidenceItems`: [{`sourceType`: "code", `sourceRef`: "src/auth/session.ts:42-87", `claim`: "Current session middleware uses express-session with Redis store", `confidence`: 0.99}, {`sourceType`: "doc", `sourceRef`: "docs/api-v2.3-mobile.md", `claim`: "Mobile clients send session cookie in X-Session-Token header", `confidence`: 0.95}]. It also identifies: `missingEvidence`: ["No documentation on whether mobile v2.3 clients can handle Authorization: Bearer headers"]—a critical gap that must be resolved before MIR finalization.

4.6 MIR — Machine Intent Representation

The MIR is the central compilation artifact—what we call the "full-set model." It must simultaneously cover goals, problem definition, success/failure criteria, constraint references, evidence references, assumptions, unresolved ambiguities, known unknowns, risk level, and routing signals. Early versions tried a "thin MIR" with just `primaryGoal` + `successCriteria` + `riskLevel`, but this caused downstream nodes to repeatedly re-guess context that should have been settled at compilation time.

```
1 interface MIR {
2   meta: ProtocolMeta;
3   taskType: string;
4   primaryGoal: string;
5   secondaryGoals: string[];
6   problemStatement: string;
7   successCriteria: string[];
8   failureCriteria: string[];
9   requiredArtifacts: string[];
10  forbiddenActions: string[];
11  resolvedAmbiguities: string[];
12  unresolvedAmbiguities: string[];
13  knownUnknowns: string[];
14  assumptions: Assumption[]; // {statement, basis, confidence}
15  constraintRefs: string[];
16  evidenceRefs: string[];
17  riskLevel: "low" | "medium" | "high" | "critical";
18  routingSignals: {value: number; risk: number;
19                  cost: number; uncertainty: number};
20  recommendedRoute: "fast" | "standard" | "deep" | "audit";
21  escalationNeeded: boolean;
22 }
```

One invariant deserves emphasis: `riskLevel {high, critical} escalationNeeded = false` is an illegal state. High-risk tasks must escalate. Early testing showed that models will confidently handle high-risk tasks without escalation if given the option—they lack the meta-cognitive awareness to recognize when they are out of their depth.

Scenario Example: The compiled MIR for the JWT task reads: `primaryGoal`: "Replace session-cookie authentication with JWT while maintaining v2.3 mobile backward compatibility", `problemStatement`: "Dual-auth transition period introduces security surface expansion", `successCriteria`: ["All new clients authenticate via JWT", "v2.3 mobile clients continue functioning without code changes"], `failureCriteria`: ["Any existing mobile session breaks", "Token leakage via logs"], `riskLevel`: "high", `escalationNeeded`: true, `routingSignals`: {value: 0.8, risk: 0.7, cost: 0.5, uncertainty: 0.4}, `recommendedRoute`: "deep". The high risk level combined with the security implications triggers mandatory escalation to the core arbiter.

4.7 ECP — Execution Contract Protocol

The ECP serves simultaneously as a control contract and an action contract. An early design mistake was splitting these into separate documents—a "what you can do" contract

and a "how the system controls you" contract. This caused nodes to satisfy one while violating the other. The merged ECP eliminates that failure mode at the cost of a larger object:

```

1 interface ECP {
2   meta: ProtocolMeta;
3   constitutionPackRefs: string[];
4   activeRoles: NodeRole[];
5   requiredCouncilSteps: CouncilStep[];
6   requiredPhases: PhaseKey[];
7   allowedActions: string[];
8   requiredActions: string[];
9   forbiddenActions: string[];
10  inputConstraints: string[];
11  outputConstraints: string[];
12  validationRequirements: string[];
13  outputRequirements: {
14    format: "markdown" | "json" | "code" | "patch" | "report";
15    mustInclude: string[];
16    mustNotInclude: string[];
17  };
18  stopConditions: string[];
19  escalationRules: Array<{when: string; then: string}>;
20  budget: {budgetTier: BudgetTier; maxDepth: number;
21    maxDelegations: number; maxCoreCalls: number};
22 }

```

Scenario Example: The ECP for the JWT task specifies: `activeRoles: ["semantic_compiler", "dissector_ant", "constraint_ant", "evidence_scout_ant", "verifier_ant", "counterexam", "core_arbiter"]`, `requiredPhases: ["parse", "design", "implement", "validate", "secure"]`, `forbiddenActions: ["Delete existing session middleware before migration complete", "Expose JWT secret in client-side code"]`, `validationRequirements: ["All existing integration tests pass", "Security audit of token lifecycle"]`, `outputRequirements: {format: "code", mustInclude: ["migration script", "rollback procedure"], mustNotInclude: ["hardcoded secrets"]}`. This contract binds all participating nodes to specific actions, gates, and output standards.

4.8 MHE — Machine Handoff Envelope

The MHE enables high-trust inter-node communication. Its most important fields are not the obvious ones (summary, payload) but the less glamorous ones: `doNotRepeat` (prevents redundant token expenditure on information already established), `knownGaps` (explicitly communicates what the sender could not determine), and `rollbackHint` (tells the receiver what to undo if things go wrong). These fields emerged from observing that nodes without them would re-read files already excerpted, re-derive facts already confirmed, and have no idea how to recover from failures.

```

1 interface MHE<TPayload = unknown> {
2   meta: ProtocolMeta;
3   fromRole: NodeRole;
4   toRole: NodeRole;

```

```

5  councilStep?: CouncilStep;
6  phase: PhaseKey;
7  summary: string;
8  confirmedFacts: string[];
9  openQuestions: string[];
10 knownGaps: string[];
11 evidenceRefs: string[];
12 payload: TPayload;
13 requestedAction: string;
14 outputContract: string[];
15 doNotRepeat?: string[];
16 rollbackHint?: string[];
17 confidence?: number;
18 }

```

Scenario Example: When the `evidence_scout_ant` completes its work and hands off to the `verifier_ant`, it produces an MHE: `{fromRole: "evidence_scout_ant", toRole: "verifier_ant", phase: "validate", summary: "Gathered 4 evidence items on current auth implementation; 1 critical gap on mobile header support", confirmedFacts: ["Session middleware uses Redis", "Mobile v2.3 sends cookie via custom header"], openQuestions: ["Can mobile v2.3 parse Bearer tokens?"], knownGaps: ["No load test data for dual-auth period"], evidenceRefs: ["ebp-001#item", "ebp-001#item-2"], requestedAction: "Verify evidence sufficiency and flag if gap blocks implementation", doNotRepeat: ["Do not re-read session.ts already excerpted in EBP"]}`. The `doNotRepeat` field prevents redundant token expenditure.

5 Runtime Execution Architecture (V2/V2.1)

5.1 From Static Architecture to Dynamic Civilization

V1 proved the protocol backbone could be defined coherently. V2 (late 2024) addressed the next question: can this backbone actually run under real task pressure? It turned out that four interlocking execution loops were needed:

1. Online Execution Loop: Direct task processing through the compilation-routing-execution-verification chain.
2. Research Assimilation Loop: Paper pool → classify/dedup → mechanism extraction → compatibility check → adversarial review → sandbox → upgrade proposal.
3. Teacher Distillation Loop: Self-retrieval → teacher consultation → critique/rebuttal → distillation record → student update proposal.
4. Version Breeding Loop: Candidate generation → offline eval → adversarial test → shadow run → gray rollout → promotion/quarantine/discard.

5.2 PromptOps: Prompt Engineering as Compilable Infrastructure

V2.1 (early 2025) addressed a problem that only became visible after V2 was running: prompt engineering was the system's weakest link. Everything else was formalized—

protocols, contracts, verification—but the actual prompts driving node behavior remained ad-hoc craft. The solution was to elevate prompts to first-class versioned assets:

- **PromptProgram**: Defines the reasoning program for a specific node on a specific task type. Not a disposable string—a versioned, evaluable, replaceable asset.
- **ContextPackingPolicy**: Defines head/tail anchoring, mid-section payload arrangement, reminder insertion rules, and conversation checkpoint triggers.
- **ConversationCheckpoint**: Records stage-wise compression results in long conversations, solving working memory hygiene and multi-turn task drift.

This choice has a cost: every prompt change now requires a version bump and evaluation pass, which slows iteration. The tradeoff is worth it for production stability but would be burdensome during early prototyping.

5.3 Tool Organs: The Model + Deterministic Tool Composite

Nodes are not pure LLM instances. Each node is a composite: a model component paired with deterministic tool organs.

- Tool organs handle: arithmetic, validation, rule matching, diff computation, parsing, retrieval, and statistics.
- Models handle: judgment, scheduling, arbitration, and creative synthesis.

Each node declares a **ToolCapabilityProfile** and operates under a **ToolCallContract**. The naming "tool organs" (rather than "tools" or "plugins") is deliberate—these are constitutive parts of the node, like organs in a body. A node without its tool organs is incomplete, not merely less capable.

5.4 Frontier Teacher Layer

The teacher layer redefines frontier API models (GPT-4, Claude, etc.) as teachers and high-level arbitrators rather than default workers. The production system first self-retrieves, self-organizes, and forms protocol packages, then submits high-purity MIR/ECP/EBP/-conflict packages to the teacher for advanced judgment.

Teacher outputs enter a **TeacherSessionRecord** and must pass verifier/counterexample/permission checks before entering the distillation chain. This was a hard-won lesson: early designs treated teacher outputs as authoritative, which created a single point of failure where teacher hallucinations propagated unchecked into the system's knowledge base. The principle now is simple—the teacher is not truth.

5.5 Forty High-Capability Nodes

V2 specifies 40 high-capability machine-language nodes organized into functional clusters. Not 40 natural-language chatbots—40 protocol-communicating execution units with formal contracts. The number emerged from mapping the minimum set of distinct cognitive functions needed to cover the full compilation-execution-verification chain without role overloading. Key node roles:

```

1 type NodeRole =
2   | "entry_ant" | "semantic_compiler"
3   | "dissector_ant" | "constraint_ant"
4   | "evidence_scout_ant" | "evidence_judge_ant"
5   | "compression_ant" | "verifier_ant"
6   | "counterexample_ant" | "risk_auditor"
7   | "queen_router" | "core_arbiter"
8   | "final_output_auditor" | "synthesizer";

```

5.6 Four Orthogonal Process Structures

ASA maintains four process structures that are vertically orthogonal—not alternatives but concurrent mechanisms at different abstraction levels:

1. Six-Step Council Protocol (macro deliberation): Proposal → Consultation → Attack/Defense → Execution → Critique → Finalization.
2. Ten-Phase Platonic Chain (task lifecycle): Parse → Depend → Design → Implement → Validate → Fault → Optimize → Secure → Version → Closure.
3. Node State Machine (runtime states): pending → running → blocked → completed → rolled-back.
4. Route Depth Policy (path depth): Fast / Standard / Deep / Audit.

The initial instinct was to unify these into a single process model. That attempt failed repeatedly—each time one structure was merged into another, edge cases appeared where the merged model could not express a valid system state. The four survived because they are genuinely orthogonal: you can be in council step "attack/defense," lifecycle phase "validate," node state "running," on a "deep" route, and all four coordinates are independently meaningful.

5.7 Research Assimilation: The 570,000-Paper Pool

V2.1 introduces a systematic research absorption mechanism for a corpus of 570,000+ papers. The pipeline operates under strict institutional constraints—the opposite of "read paper, immediately apply idea":

1. Ingestion and Deduplication: Papers enter the pool, are classified by domain relevance, and deduplicated against existing mechanism records.
2. Mechanism Extraction: Each paper is reduced to a `PaperMechanismRecord`—not a summary, but a formal extraction of the structural mechanism that could benefit routing, memory, anomaly defense, task gating, error correction, or resource allocation.
3. Compatibility Check: Extracted mechanisms are evaluated against the existing protocol backplane for compatibility. Incompatible mechanisms are flagged but not discarded.

4. Adversarial Review: Candidate mechanisms undergo adversarial critique—counterexample nodes attempt to find failure modes, edge cases, or conflicts with existing iron laws.
5. Sandbox Experiment: Surviving mechanisms enter a sandboxed experimental environment with feature flags, isolated from production.
6. Upgrade Proposal: Only after sandbox validation does a mechanism generate a formal `UpgradeProposal` linking to the version breeding pipeline.

The principle is absolute: paper pools do not equal direct capability growth. A paper must be ingested, purified, adversarially tested, sandboxed, and formally proposed before it can influence production behavior. This pipeline is deliberately slow—the cost is latency between "interesting paper published" and "mechanism available in production." The benefit is that the system never absorbs a mechanism that breaks existing invariants.

5.8 Route Scoring and Budget Allocation

The Queen Router computes a `RouteScore` based on four primary signals and optional modulation:

Primary signals:

- Value (V): How important is the task outcome? High-value tasks justify deeper routes.
- Risk (R): Potential for harm from errors. High-risk triggers escalation.
- Cost (C): Computational budget available.
- Uncertainty (U): How much is unknown? High uncertainty favors deeper exploration.

V3 added modulation signals (K/T/P/M) in 2025:

- K (Knowledge coverage): Does the system already have relevant knowledge cached?
- T (Tool determinism): Can tool organs handle this deterministically?
- P (Prompt stability): Is the `PromptProgram` for this task type well-tested?
- M (Model fitness): How well does the assigned model match this task profile?

Budget tiers (B0–B3) map to increasing resource allocation. The initial design used only V/R/C/U. The K/T/P/M signals were added after observing that two tasks with identical V/R/C/U profiles could have very different optimal routes depending on whether the system had prior experience with that task type.

5.9 Automatic Rollback and Cost Routing

V2 introduces formal rollback levels rather than the naive "rerun everything" approach. The graduated design came from observing that most failures are local (a single node produced bad output) but default recovery in existing systems is global (restart the entire pipeline):

- Level 0: Node-local retry with adjusted parameters.
- Level 1: Roll back to the beginning of the current phase.
- Level 2: Return to the Queen Router for re-routing with updated risk signals.
- Level 3: Return to the semantic compiler with flagged compilation errors.
- Level 4: Surface to the user with a structured explanation of what failed and why.

Each level has explicit trigger conditions and cost implications. The system prefers the cheapest recovery that addresses the actual failure mode. There is a known weakness here: determining the correct rollback level requires diagnosing the failure cause, which is itself a non-trivial inference task. Currently this diagnosis is handled by the verifier node, but misdiagnosis (rolling back too far or not far enough) remains an observed failure mode.

6 Civilization-Level Continuous Evolution (V3)

6.1 Four-Domain Structure

V3 (mid-2025) addressed a problem that V2 surfaced but could not solve: the system needed to simultaneously run production tasks, learn from experience, absorb external research, and evolve its own components—without these activities interfering with each other. The solution was strict four-domain separation:

1. Governance and Legal Domain: Mother law, anchor tables, Clause Registry, clause graphs, role law slices, task law overlays, runtime constitution packs, and unconstitutionality audit records. This domain answers not "how to do tasks" but "under what legal system does the entire civilization operate."
1. Online Production Execution Domain: The direct task-processing main chain including HPS, semantic compiler, PromptProgram, ContextPackingPolicy, the full protocol suite, RouteDecision, node society, tool organs, verification chain, core arbiter, and final output auditor.
1. Offline Learning and Research Domain: Absorbs paper pools, code repositories, documentation, runtime traces, teacher outputs, and verifier/counterexample logs. Transforms them into mechanism objects, training data, preference signals, routing signals, and upgrade proposals. Principle: absorb first, purify second; adversarial first, train second; sandbox first, candidate second; offline first, gray-release second.
1. Version Breeding and Control Domain: Manages stable/candidate/shadow/quarantined version pools, migration, lineage, offline evaluation, adversarial testing, gray rollout, and promotion/quarantine/discard decisions. The goal is not "self-replacement" but "self-breeding"—new versions cannot self-promote; they must pass multi-metric evaluation, adversarial testing, shadow running, and gray-scale replacement with rollback capability.

The domains share governance, protocols, and audit infrastructure, but maintain strict boundaries. The most important boundary: research domain outputs cannot enter the production domain except through a formal transit package. This constraint was later elevated to an iron law after an incident where an untested routing heuristic leaked from research into production and caused a cascade of suboptimal route decisions.

6.2 Twelve Formal Planes

V3 establishes twelve formal planes as the stable architectural backbone. Any new layer must fit within one of these twelve planes; otherwise it is not permitted to float at the architecture level. This constraint was introduced after V2's development saw several "orphan layers" that belonged to no clear plane and created maintenance confusion:

1. Constitutional Governance Plane — Mother law, anchors, registry, clause priority, role applicability, violation actions, runtime constitution packs.
2. Semantic Compilation Plane — $HPS \rightarrow ISG \rightarrow CRM \rightarrow EBP \rightarrow MIR \rightarrow ECP \rightarrow MHE$ main chain plus context orchestration: head/tail anchoring, mid-section payload, reminder insertion, conversation checkpoints.
3. Protocol Backplane — Unified communication bus for all typed protocol objects; the machine-language communication backbone shared by high-capability nodes.
4. PromptOps Plane — Prompt strategies compiled into system assets: PromptProgram, ContextPackingPolicy, ConversationCheckpoint, PromptEvaluationRecord, Operator Prompt Library.
5. Tool Organ Plane — Institutionalizes "model + deterministic tool organ" composites with ToolCapabilityProfile and ToolCallContract per node.
6. Node Society Plane — Preserves role-based execution with formal boundaries, state machines, tool organs, prompt programs, escalation and rollback conditions per node.
7. Route and Gate Plane — RouteScore, RouteDecision, budget tiers, depth selection, escalation gates, plus thalamic gating and basal-ganglia-style action selection with K/T/P/M modulation signals.
8. Verification and Immune Plane — Verifier, counterexample, final auditor, schema/zod assertions, risk register, plus immune-style anomaly defense detecting fabricated facts, reasoning breaks, constraint forgetting, attention decay, and protocol drift.
9. Frontier Teacher Plane — Defines API/frontier models as teachers and high-level arbitrators. Production systems self-retrieve and self-organize before submitting high-purity packages to teachers.
10. Training Civilization Plane — Upgrades training from single-model to model-society: Dataset Factory, role-specific models, route scorer, escalation predictor, budget allocator, reward/preference model, verifier trigger model, tool-use policy, distillation chain.

11. Research Assimilation and Bio Sandbox Plane — Paper pool ingestion, deduplication, mechanism extraction, compatibility judgment, adversarial review, sandbox experiments, and bio/brain-inspired experiments under protocol constraints.
12. Version Breeding and Control Plane — CandidateVersion, UpgradeProposal, Lineage, Migration, Offline Eval, Adversarial Test, Shadow Run, Gray Rollout, Promotion/Quarantine/Discard.

6.3 Five Concurrent Closed Loops

V3’s capability derives from five simultaneously operating closed loops. A system that only executes tasks is brittle; one that also learns, researches, breeds versions, and manages long conversations can improve without human intervention—provided each loop is independently governed:

1. Task Execution Loop: HPS → semantic compilation → PromptProgram/ContextPacking → MIR/ECP → RouteDecision → Node Graph + Tool Organs → Verifier/Counterexample → Core Arbiter/Teacher → Final Auditor → Delivery → Audit Log.
1. Long Conversation Management Loop: Conversation turns → WorkingMemory → Checkpoint → Re-anchor → Continue/Branch/Hard Reset. Prevents goal drift, constraint loss, and attention decay across multi-turn interactions.
1. Teacher Learning Loop: Self-Retrieval → Teacher Consultation → Teacher Critique/Rebuttal → Distillation Record → Student Update Proposal.
1. Research Assimilation Loop: Paper Pool → Classify/Dedup → Mechanism Extraction → Compatibility Check → Adversarial Review → Sandbox Experiment → Upgrade Proposal.
1. Version Breeding Loop: Candidate Generation → Offline Eval → Adversarial Test → Shadow Run → Gray Rollout → Promotion/Quarantine/Discard → Migration.

6.4 V3 Iron Laws

V3 establishes fifteen non-negotiable iron laws. Among them:

- Research cannot directly enter production.
- Stable versions cannot self-overwrite.
- High-value nodes default to consuming MIR/ECP/evidence only, not raw HPS.
- Internal communication must be protocol-typed.
- Deterministic problems go to tool organs first; models handle judgment only.
- PromptProgram is a first-class asset.
- The teacher is not truth.

- Paper pools do not equal direct capability growth.
- Bio-inspired layers can only grow in the research domain first.
- All upgrades must preserve lineage.
- Failure does not equal "rerun everything."
- The training target is the model society, not a single omnipotent model.

6.5 V3 Ten New First-Class Objects

V3 introduces exactly ten new first-class protocol objects that extend the V1/V2 canonical schema:

Object	Responsibility
PromptProgram	Defines the reasoning program for a node on a task type; versioned, evaluable, reproducible
ContextPackingPolicy	Head/tail anchoring, mid-section payload, reminder insertion, summary checkpointing
ConversationCheckpoint	Stage-wise compression for long conversations; working memory hygiene
ModelProfile	Capability characterization per model: strengths, weaknesses, cost, latency, hallucination rate
ToolCapabilityProfile	Declares what deterministic operations a node's tool organs can perform
TeacherSessionRecord	Full audit trail of teacher consultations: input, output, critique, permission status
DistillationRecord	Tracks what was distilled from teacher to student: source, target, validation status
PaperMechanismRecord	Formal extraction of a paper's structural mechanism with compatibility assessment
UpgradeProposal	Connects research domain to version breeding: source, evidence, affected modules
CandidateVersion	Supports stable/candidate/shadow/quarantined version pool with promotion criteria

6.6 Bio-Inspired Experimental Mechanisms

V3's Research Assimilation and Bio Sandbox Plane incorporates six bio/brain-inspired experimental mechanisms, all under strict protocol constraints (feature-flagged, sandboxed, auditable, reversible):

1. Immune-Style Anomaly Defense: Self/non-self discrimination for detecting fabricated facts, reasoning breaks, and protocol drift. Maintains immune memory of repeated failure patterns.
2. Thalamic Gating: Attention-filtering mechanism that gates information flow to high-value nodes, preventing cognitive overload and irrelevant signal propagation.
3. Hippocampal Memory Indexing: Episodic memory organization for rapid retrieval of relevant past experiences, route priors, and failure patterns.
4. Basal Ganglia Action Selection: Competitive inhibition mechanism for action selection in the Queen Router—competing route candidates inhibit each other based on signal strength.
5. Cerebellar Refinement: Error-correction mechanism that fine-tunes node outputs based on prediction-error signals from the verification chain.

6. Ant Pheromone-Like Route Statistics: Accumulated route success/failure statistics that bias future routing decisions, analogous to pheromone trail reinforcement in ant colonies.

A note on intellectual honesty: these bio-inspired mechanisms are speculative. They are included as formal experimental candidates, not as proven components. Each must demonstrate measurable improvement over purely algorithmic baselines before promotion to production. The immune-style anomaly defense has shown the most promise in early testing; the cerebellar refinement mechanism has proven difficult to implement without introducing training instability. The pheromone-like route statistics work well for frequently-encountered task types but degrade for novel tasks where no prior statistics exist.

6.7 V3 Phased Implementation Priority

V3 enforces strict ordering to prevent premature expansion:

- Phase V3-A: Protocol and governance closure — RouteDecision formalization, ClauseMap/-SourceAnchorMap/MigrationContract, package completion criteria and gate suites.
- Phase V3-B: PromptOps and Tool Organ formalization — immediate production stability gains.
- Phase V3-C: Teacher Layer and Distillation — "self-retrieve → consult → rebut → distill → upgrade" pipeline.
- Phase V3-D: Research Assimilation — paper ingestion through upgrade proposal.
- Phase V3-E: Version Breeding — stable/candidate/shadow/quarantined pool with gray rollout.
- Phase V3-F: Tier-1 Bio Experiments — all feature-flagged, sandbox-first.

7 Autonomous Self-Governance (V4)

7.1 Positioning

V4 (January–February 2026) shifts the system's center of gravity from "making the system stronger" to "making the system safely change itself." The question that forced V4 into existence was straightforward: V3's five closed loops all produce upgrade proposals, but who decides which proposals are safe to adopt? Without V4, the answer was "the developer reviews everything manually"—which defeats the purpose of autonomous evolution.

The one-sentence version: V3 makes the civilization come alive; V4 makes it learn how to safely change itself.

#	Problem	Core Proposition
1	Autonomy Strength	Which changes can auto-execute, which need dual-gate, which need hu
2	Research-Production Switching	Research outputs enter production via formal transit packages with co
3	Civilization Control Plane	The control plane observes the entire civilization: autonomy mode, tea
4	Cross-Version Continuity	After upgrades: how to maintain continuity of legal system, memory, m
5	Meta-Governance	Who can modify autonomy boundaries, promotion institutions, high-s

7.2 Five Hard Problems

7.3 Five New Formal Planes

1. Meta-Governance Plane: Governs constitutional amendment proposals, autonomy boundary revision proposals, and high-sensitivity gate change proposals. It does not govern tasks—it governs how the system is allowed to change the system. This is the slowest-moving plane in the entire architecture by design.
1. Autonomy Envelope Plane: Defines auto-executable actions, dual-gate actions, human-approval actions, permanently forbidden actions, autonomy ceilings per mode, and emergency brake conditions. The "permanently forbidden" category exists because some actions (e.g., modifying the mother law without human approval, deleting audit logs, bypassing the counterexample chain) should never be automatable regardless of how confident the system becomes.
1. Research-Production Switching Plane: Packages research outputs into `ResearchTransitPackage` objects that pass through compatibility gates, shadow runs, gray rollouts, and rollback contracts before entering production. This is the formal solution to the "research leakage" problem observed in V3.
1. Civilization Control Plane: Provides global state snapshots, risk posture, autonomy strength, teacher dependency, version pool health, and critical civilization metrics—observing the civilization rather than just tasks.
1. Continuity and Heritage Plane: Maintains a `HeritageLedger` specifying which memories, rules, route priors, immune memories, and experiences are inherited, compressed, archived, or explicitly forgotten. The "explicit forgetting" mechanism was controversial—it means the system can decide to discard information. But without it, memory accumulates without bound and eventually degrades retrieval quality.

7.4 Core Protocol Objects

- `AutonomyEnvelopePolicy`: Auto-executable actions, dual-gate actions, human-approval actions, permanently forbidden actions, autonomy ceilings, autonomy contraction rules, kill-switch conditions.
- `ResearchTransitPackage`: Source mechanism, experiment references, verifier/counterexample results, compatibility checks, candidate insertion points, rollback plans, affected modules.

- **PromotionContract**: Minimum shadow duration, gray-scale ratio, blocking conditions, rollback triggers, required lineage conditions.
- **ControlPlaneSnapshot**: Stable version, candidate pool summary, autonomy mode, teacher load, research pipeline status, high-risk subsystems, health metrics.
- **HeritageLedger**: Rule lineage, retained memories, retired memories, route priors, immune failure memories, explicit forgetting decisions, heritage inheritance records.
- **CapabilityMap**: Strong domains, weak domains, unstable domains, teacher-dependent domains, tool-dependent domains, high-hallucination-risk zones, low-confidence route families.

The **CapabilityMap** is the system’s formal self-model. Without it, autonomy boundaries cannot be calibrated—the system cannot know where it is safe to act autonomously if it does not know where it is competent. Building an accurate self-model is itself an unsolved problem; the current approach uses accumulated verification pass/fail rates per task type as a proxy for competence.

7.5 Seven Iron Laws of V4

1. Autonomy must be bounded—the system can self-optimize but cannot self-modify without boundaries.
2. Research entering production must be transactional: formal transit packages, promotion contracts, rollback plans.
3. Stable version identity cannot silently rupture.
4. Meta-governance is always slower than execution governance. The higher the rule level, the slower the change cadence.
5. All automatic research absorption, teacher distillation, candidate promotion, and memory consolidation must be visible to the control plane.
6. Reversibility trumps spontaneity. The truly valuable capability is recovering stably from errors, not changing fast.
7. Heritage trumps replacement—generational succession while preserving legal system and lineage, not new versions killing old ones.

8 Federated Civilizational Network (V5)

8.1 From Single Civilization to Federation

V5 (March 2026) was motivated by a practical question: what happens when two independently-developed ASA instances need to collaborate? The single-civilization model has no answer—it assumes one legal system, one version pool, one control plane. V5 adds the machinery for multi-civilization federation: sovereignty boundaries, cross-civilization treaties, knowledge exchange, federated task orchestration, trust and sanctions, disaster isolation, and civilization forking.

V4 solves "how one civilization safely changes itself"; V5 solves "how multiple civilizations collaborate without losing sovereignty or safety boundaries."

Whether V5 is premature given that V1-V4 have not yet been deployed at scale is a legitimate concern. The counter-argument: federation constraints are easier to design before deployment than to retrofit after multiple independent instances have already diverged.

8.2 Generational Evolution Gradient

Generation	Core Breakthrough	One-Sentence Positioning
V1	Protocol backbone	Compress natural language tasks into governable protocol objects
V2/V2.1	Runtime + Learning	Make the system run, and begin research absorption, teacher o
V3	Civilization-level evolution	Weave execution, learning, research, breeding into one legal sy
V4	Autonomous safe self-change	Make a single civilization learn to safely change itself within in
V5	Federated civilization network	Make multiple civilizations collaborate under sovereignty prese

8.3 Six Hard Problems

1. Civilization Sovereignty: Which knowledge, memories, rules, and upgrade proposals can flow across civilizations, and which can never be cross-border copied. Without this distinction, federation degrades into a single merged system—defeating the purpose.
1. Cross-Civilization Communication: Civilizations cannot collaborate via free-form natural language. They rely on treaty objects, export packages, attestations, and rollback clauses. This mirrors the same insight driving the compilation pipeline: unstructured communication loses information and accountability.
1. Trust and Liability Attribution: Cross-civilization outputs must answer: which civilization produced this, what is its reputation grade, did it pass verification, and how is liability traced if it contaminates the federation.
1. Federated Task Orchestration: RouteDecision’s unit expands from nodes to civilizations. Research civilizations, production civilizations, security civilizations may jointly participate in a federated task.
1. Disaster Isolation: The design assumes partial civilizations will lose control, degrade, or propagate erroneous route priors. This is a normal operating condition, not an exception. Civilization-level isolation and circuit-breaking are required.
1. Civilization Forking and Inheritance: When a civilization legally forks, which legal systems and capabilities are inherited vs. severed, and when re-merging is allowed—all must be institutionalized.

8.4 Six New Formal Planes

1. Sovereignty Plane: Civilization-level boundaries, federal/local legal system layering, shareable vs. non-shareable asset classification, autonomy ceiling inheritance rules.
1. Treaty Plane: Cross-civilization treaties, acceptable inputs, exportable knowledge scope, liability boundaries, rollback clauses, sanctions clauses, validity periods.
1. Federation Routing Plane: Extends routing unit from node to civilization. Civilization-level task dispatch, division of labor, cost sharing, and fallback.
1. Trust and Reputation Plane: Civilization reputation, proposal credibility, output distortion history, liability tracking, sanction records, intra-federation trust gradients.
1. Civilizational Isolation and Recovery Plane: Quarantine, federation circuit-breaking, civilization-level rollback, blacklist/watchlist, disaster recovery, kill switch.
1. Forking and Inheritance Plane: Civilization forking, inheritance rules, memory/legal-system/capability lineage, fork evaluation, re-merge, and independence promotion.

8.5 Core Protocol Objects

```
1 interface SovereigntyProfile {
2     governingConstitutionRefs: string[];
3     sharedLawRefs: string[];
4     shareableAssets: string[];
5     exportRestrictions: string[];
6     importRestrictions: string[];
7     maxAutonomyLevel: number;
8 }
9
10 interface TreatyContract {
11     parties: string[];
12     acceptedObjectTypes: string[];
13     prohibitedTransfers: string[];
14     attestationRequirements: string[];
15     liabilityRules: string[];
16     rollbackClauses: string[];
17 }
18
19 interface FederationRouteDecision {
20     participants: string[];
21     civilizationRoles: Record<string, string>;
22     splitPlan: string;
23     costSharePlan: string;
24     trustAdjustments: Record<string, number>;
25     fallbackCivilizations: string[];
26 }
27
28 interface TrustAttestation {
```

```

29 sourceCivilization: string;
30 verifierRefs: string[];
31 counterexampleRefs: string[];
32 trustGrade: number;
33 liabilityClass: string;
34 decayPolicy: string;
35 }
36
37 interface CivilizationForkRecord {
38     parentCivilization: string;
39     childCivilization: string;
40     inheritedAssets: string[];
41     divergencePurpose: string;
42     remergePolicy: string;
43     promotionPath: string;
44 }

```

8.6 Eight Iron Laws of V5

1. Federation cooperation cannot cancel sovereignty.
2. Civilizations can only cooperate through treaties—high-value cross-civilization collaboration must be treaty-bound.
3. Federation routing must carry trust gradients.
4. Single-civilization failure cannot drag down the federation. Partial failure is a design-time assumption.
5. Civilization forking must be legalized as a formal lineage event.
6. Cross-civilization knowledge exchange must be auditable, traceable, and revocable.
7. Federation continuity trumps single-civilization convenience.
8. The federation control plane must observe civilization ecology: growth, teacher dependency, contaminated outputs, fork incubation, treaty expiration.

9 Civilizational Economics and Constitutional Economy (V6)

9.1 Motivation

V6 (March–April 2026) emerged from a specific observation: once V5’s federation was sketched out, it became clear that sovereignty and treaties alone cannot sustain long-term multi-civilization collaboration. Six problem classes surfaced:

1. Resource Scarcity: Compute, teacher budgets, research quotas, audit resources, and memory bandwidth are finite goods requiring economic allocation. V5 assumes infinite resources; reality does not.

2. Capability Asymmetry: Civilizations specialize. Capability exchange requires licensing, pricing, liability, and revocation rules.
3. Externalities: A civilization pursuing local efficiency may impose federation-level costs—over-consuming teacher models, flooding immature proposals, creating systemic fragility through monoculture.
4. Precedent and Arbitration: Long-term collaboration requires accumulated jurisprudence. How were disputes previously resolved? Which capability exchanges historically failed?
5. Ecological Monoculture: If the federation only rewards "cheapest/fastest," diversity collapses. All civilizations converge on the same route and teacher dependency—efficient short-term, catastrophically fragile long-term.
6. Macro Strategy: The federation must allocate resources between research and production, decide which forks to nurture or terminate, address ecological overheating.

9.2 Six New Formal Planes

1. Civilizational Economy Plane: Civilization-level resource accounting, cost/benefit attribution, compute/teacher/audit/research pricing, federation-level value exchange.
1. Capability Exchange and Licensing Plane: Role-model capability export, tool organ capability export, teacher knowledge sharing, research mechanism licensing, cross-civilization capability use permits and revocation.
1. Federal Treasury and Budget Plane: Federation resource pool, high-risk task subsidies, research/production investment allocation, teacher model total budget control, civilization health repair fund.
1. Jurisprudence and Arbitration Plane: Dispute classification, precedent retrieval, cross-civilization liability rulings, capability licensing dispute arbitration, sanction escalation logic, treaty interpretation and conflict adjudication.
1. Ecology and Diversity Plane: Monitor civilization monoculture, capability monopoly, teacher dependency concentration, route family convergence. Maintain federation-level diversity and resilience.
1. Macro Planning and Scenario Simulation Plane: Federation strategic simulation, civilization combination stress testing, long-term research/production/security resource allocation, fork investment and termination strategy, federation future state projection.

9.3 Core Protocol Objects

- CivilizationalBalanceSheet: Compute assets, teacher budget, audit capacity, memory capacity, trust liabilities, sanctions exposure, research reserves.

- **CapabilityLicense**: Capability type, source/target civilization, permitted scope, revocation conditions, liability sharing, audit requirements.
- **FederalTreasuryAllocation**: Target civilizations, allocation purpose, expected return, risk class, review window, rollback/freeze triggers.
- **PrecedentCaseRecord**: Dispute type, involved civilizations, fact summary, applied treaty/law refs, ruling, rationale, precedent weight, future applicability.
- **ExternalityReport**: Source civilization, affected domains, externality type, measured damage, remediation plan, compensation/sanction proposal.
- **EcologyHealthSnapshot**: Diversity index, teacher concentration index, route monoculture risk, capability monopoly risk, fork vitality, systemic fragility score.
- **ScenarioPortfolio**: Scenario set, resource assumptions, civilization growth assumptions, stress tests, recommended strategic posture, early warning triggers.

9.4 Eight Iron Laws of V6

1. Markets cannot override the constitution. The economy plane serves civilization; it does not rule it. This is the single most important constraint in V6.
2. Capability export must carry licenses—no cross-civilization capability use without authorization.
3. Externalities must be visible, measurable, and accountable.
4. Diversity trumps local optimality. The federation cannot force all civilizations onto the same route for short-term efficiency.
5. Treasury is slower than routing. Civilization fiscal allocation must be slow to prevent systemic instability.
6. Precedent is slower than treaty; treaty is slower than constitution. This hierarchy cannot be inverted.
7. If one civilization’s expansion significantly increases federation fragility, it should not be infinitely amplified even if locally stronger.
8. Macro planning, fiscal allocation, and fork investment must all carry rollback/freeze/quarantine conditions.

9.5 The Constitutional Economy Model in Detail

V6’s economic model is not a free market but a constitutional economy—economic activity bounded by constitutional law, market mechanisms serving civilizational goals rather than overriding them.

Resource Scarcity Formalization:

In a federation of N civilizations, the following resources are formally scarce and require economic allocation:

- Compute cycles (inference budget per civilization per epoch)

- Teacher model consultation quota (frontier API calls)
- Audit capacity (verifier/counterexample node availability)
- Memory bandwidth (long-term memory storage and retrieval)
- Research slots (concurrent sandbox experiments)

Each civilization maintains a **CivilizationalBalanceSheet** that tracks assets (accumulated compute credits, research reserves, trust capital) against liabilities (sanctions exposure, externality debts, teacher dependency costs).

Capability Exchange Mechanics:

When Civilization A needs a capability that Civilization B possesses (e.g., a specialized verifier for medical claims), the exchange follows a formal protocol:

1. A issues a **CapabilityLicenseRequest** specifying scope, duration, and offered compensation.
2. B evaluates against its **SovereigntyProfile** to confirm the capability is exportable.
3. A **CapabilityLicense** is issued with: permitted scope, revocation conditions, liability sharing rules, and audit requirements.
4. Usage is metered and recorded in both civilizations' balance sheets.
5. Revocation triggers (quality degradation, sovereignty violation, treaty breach) are continuously monitored.

Externality Accounting:

V6 introduces formal externality accounting—a mechanism to prevent the tragedy of the commons:

- Positive externalities (a civilization's research benefits the federation) are credited.
- Negative externalities (over-consuming shared teacher budget, outputting low-quality route priors that pollute other civilizations) are debited.
- **ExternalityReport** objects accumulate into a civilization's reputation score, influencing future **FederationRouteDecision** trust adjustments.

Jurisprudence Accumulation:

The Jurisprudence and Arbitration Plane operates on three temporal scales:

- Instant arbitration: Ad-hoc dispute resolution for novel conflicts (no precedent exists).
- Precedent formation: When similar disputes recur 3+ times, a **PrecedentCaseRecord** is formalized with ruling rationale and future applicability weight.
- Constitutional interpretation: When precedents conflict or treaty language is ambiguous, constitutional-level interpretation is invoked—the slowest and most authoritative process.

This creates a legal system that grows more efficient over time as precedent accumulates, while maintaining constitutional supremacy over all economic and treaty-level decisions.

Anti-Monopoly and Diversity Mechanisms:

The Ecology and Diversity Plane actively monitors and counteracts monoculture:

- Diversity Index: Measures variety in route families, teacher dependencies, training approaches, and capability distributions across the federation.
- Concentration Alerts: Triggered when any single civilization controls >40% of a critical capability, or when >60% of civilizations share the same teacher dependency.
- Diversity Subsidies: The Federal Treasury may allocate resources to underdeveloped capability areas or minority approaches that provide systemic resilience.
- Fork Vitality Score: Measures the health of the civilization forking ecosystem—too few forks indicates stagnation; too many indicates fragmentation.

10 Architectural Invariants and Conflict Resolution

10.1 The Twelve Hardest Invariants

These twelve invariants survived every version iteration. They are the rules that, when violated even temporarily during development, caused cascading failures:

1. HPS Losslessness: The Human Prompt Snapshot must be preserved without loss. Speculation must never be written back into the original input.
2. Natural Language is Source Only: Natural language can serve as task source but cannot serve as long-term high-trust execution language.
3. High-Value Node Isolation: High-value nodes (core arbiter, synthesizer) default to consuming only MIR/ECP/critical evidence—never raw HPS.
4. Unresolved Ambiguity Blocks Execution: Unresolved ambiguities must not be signed off into implementation-phase ECPs.
5. Mandatory Escalation: High-risk tasks must escalate. `riskLevel  $\in$  {high, critical}  $\wedge$  escalationNeeded = false` is an illegal state.
6. Evidence Over Summary: Critical conclusions must have evidence. Summaries cannot substitute for evidence.
7. Universal Audit: The supervision chain is not decorative. Even the core model must be audited.
8. No Execution Shortcuts: Execution steps cannot be simplified away. No TODO, ..., or placeholder outputs.
9. Process Integrity: The six-step council and ten-phase lifecycle must be preserved—they are not stylistic decoration.

10. Cost Cannot Override Evidence: Cost savings cannot sacrifice evidence completeness or constraint completeness.
11. Experiment Before Architecture: No architecture claim is valid without experimental comparison against direct-model and simple-workflow baselines.
12. Implementation Before Publication: Build artifacts first, write papers second; internal governance documents first, external expression second.

Invariant #12 is worth noting: this paper was written after the architecture was complete, not before. The temptation to publish early was resisted because premature publication creates pressure to defend design choices that may need revision.

10.2 Conflict Resolution Rules

When conflicts arise between system components, ASA applies a strict priority hierarchy. These were established after observing that without explicit priority rules, different nodes would resolve the same conflict differently depending on which information they encountered first:

Rule Source Priority: Mother Law Original Text > Anchor Table > Clause Registry > Clause Graph > Runtime Constitution Pack > Single-Task ECP > Node Local Output.

Protocol Layer Priority: Canonical Schema > JSON Schema / Zod > Example Data > Pseudocode > Narrative Description.

Value Priority: Truthfulness & Confidentiality > Hard Constraints > Verification & Completeness > Routing & Cost > Style & Expression Preferences.

10.3 Naming Unification and Migration

ASA employs a three-layer protocol architecture to manage naming evolution—a pragmatic response to the reality that naming conventions shifted multiple times during V1-V3 development, and historical traces needed to remain valid:

- Conceptual Layer: Preserves HPS/ISG/CRM/EBP/MIR/ECP/MHE theoretical completeness.
- Runtime Layer: Uses engineering fields: taskId, traceId, PhaseKey, NodeRole, TaskRuntimeState.
- Migration Layer: Through version headers, migration records, and compatibility strategies, absorbs legacy naming without rupturing historical traces.

Role naming uses a three-level mapping:

- 20 Council Experts = parliamentary semantic role universe
- entry_ant / verifier_ant / core_arbiter = architecture-level runtime roles
- prompt-dissector / evidence-miner / synthesizer = implementation-level technical aliases

A `role_alias_map` unifies these without forced renaming. The alternative—renaming everything to a single canonical form—was attempted and abandoned because it broke every existing trace and test fixture.

11 Implementation Architecture

11.1 Package Structure

The engineering realization follows a six-group implementation sequence. Each group depends on the previous:

Group 1 — Protocol Backbone:

```
packages/protocol-types/src/  
  meta.ts, canonical-types.ts, json-schema.ts,  
  zod.ts, error-codes.ts, migration.ts
```

Group 2 — Constitutional Compilation:

```
packages/constitution/src/  
  anchor-index.ts, source-registry.ts,  
  runtime-clause-graph.ts, constitution-pack.ts,  
  role-slice-compiler.ts
```

Group 3 — Semantic Compiler:

```
packages/prompt-compiler/src/  
  hps-normalizer.ts, segmenter.ts,  
  ambiguity-detector.ts, constraint-extractor.ts,  
  evidence-aligner.ts, mir-builder.ts,  
  ecp-builder.ts, mhe-builder.ts
```

Group 4 — Runtime and Routing:

```
packages/node-runtime/src/node-state-machine.ts  
packages/queen-router/src/  
  route-score.ts, route-selector.ts, route-decision.ts  
packages/evidence-engine/src/evidence-pool.ts
```

Group 5 — Truth Maintenance:

```
packages/verifier-engine/src/  
  structural-checker.ts, semantic-checker.ts,  
  evidence-checker.ts, phase-checker.ts,  
  counterexample-engine.ts  
packages/audit-log/src/trace-writer.ts
```

Group 6 — Minimum Viable Loop:

```
packages/synthesis-engine/src/  
  final-synthesizer.ts, final-auditor.ts  
packages/experiment-kit/src/  
  baselines.ts, metrics.ts
```

After Group 6, the system has a minimum closed loop—it can accept a prompt, compile it, route it, execute it, verify it, and deliver output. Everything beyond this is enhancement, not foundation.

11.2 MVP Constraints

Permitted simplifications (deferrable without breaking invariants):

- 20 expert roles need not all be instantiated initially; only necessary role mappings are required.
- Deep and audit routes may share partial verifier implementations.
- BudgetOptimizer may use static thresholds initially.
- Control plane may start with trace viewing only.

Forbidden simplifications (cutting these breaks the architecture’s identity):

- Letting the core model directly consume HPS instead of the compilation chain.
- Deleting CRM/EBP and going directly HPS→MIR.
- Using natural language summaries instead of MHE.
- Merging the six-step council, ten-phase lifecycle, and node state machine into one crude process.
- Reducing ECP to only budget and roles without action/validation/output gates.
- Removing the counterexample chain.
- Omitting version headers, migration mechanisms, evidence refs, or rollback levels.

The first list is a backlog; the second is a set of anti-patterns.

12 Discussion

12.1 Comparison with Existing Approaches

Dimension	Traditional Multi-Agent	ASA
Inter-agent communication	Natural language	Typed protocol objects (MIR/ECP/MHE)
Governance	Ad-hoc rules or none	Constitutional compilation with clause registry
Verification	Optional post-hoc review	Mandatory adversarial verification chain
Routing	Heuristic model selection	Value/risk/uncertainty/cost scoring with budget tie
Evolution	Manual updates	Institutional version breeding with shadow/gray/pr
Federation	Not addressed	Sovereignty, treaties, trust attestation, economics
Economics	Not addressed	Balance sheets, capability licensing, treasury, prece

12.2 The Cognitive Interface Error

The observation that motivated this entire project: when a model receives a raw natural language prompt, it must simultaneously parse intent, identify constraints, gather evidence, plan execution, execute, and self-verify. Most of the inference budget goes to translating human language into execution semantics rather than to the judgment the user actually cares about. The compilation pipeline ensures that by the time the core arbiter receives input, it has already been compiled into a fully-specified MIR with resolved ambiguities, validated evidence, explicit constraints, and a clear execution contract.

Whether this overhead is justified depends entirely on task complexity. For simple queries ("what is $2+2$?"), the compilation pipeline is pure waste. For complex multi-constraint tasks with high stakes, the compilation cost is small relative to the cost of a wrong answer. The route depth policy exists precisely to modulate this tradeoff.

12.3 Scalability Considerations

The architecture scales along three dimensions. Vertical: adding more specialized nodes within a single civilization. Horizontal: federating multiple civilizations with sovereignty boundaries. Temporal: version breeding and research assimilation enable continuous improvement without identity rupture.

The protocol backplane ensures that scaling does not introduce communication ambiguity—all inter-component communication remains typed and traceable regardless of system size. An open question: whether the protocol overhead becomes a bottleneck at very high node counts (hundreds of nodes per civilization). Current design assumes tens of nodes; scaling to hundreds may require protocol compression or batching not yet specified.

12.4 Limitations and Future Work

1. Empirical Validation: This paper presents the architectural framework. Large-scale empirical comparison against baseline systems remains future work.
2. Computational Overhead: The compilation pipeline adds latency. For the fast route this is minimal; for the audit route it is substantial. Optimizing the fast-route path while maintaining protocol guarantees is an engineering problem not yet solved.
3. Constitutional Completeness: Formal verification that the mother law is internally consistent and complete is an open theoretical problem. The current approach relies on adversarial testing rather than formal proof.
4. Economic Equilibrium: V6's civilizational economics requires game-theoretic analysis to prove stability. Certain resource distributions may lead to oscillation or deadlock—this has not been formally analyzed.
5. Bio-Inspired Integration: The six bio-inspired mechanisms remain experimental. Only the immune-style anomaly defense has shown clear benefits; the others are promising but unproven.
6. Single-Developer Limitation: This architecture was developed by a single researcher. It has not been stress-tested by a team with diverse perspectives, which increases the risk of blind spots.

13 Conclusion

This paper has described the Ant Social Architecture across its six evolutionary versions, developed between early 2024 and April 2026. The core claims, stripped to their essentials:

1. Natural language is task source code, not execution language. The HPS→ISG→CRM→EBP→MIL pipeline eliminates the cognitive interface error by separating understanding from execution.
1. The mother law is not a guiding principle but a compilable source document—clause registries, runtime constitution packs, enforcement gates.
1. All inter-node communication uses typed protocol objects with full traceability. No natural language handoffs.
1. Verification is not optional post-hoc review but a mandatory architectural layer: structural, semantic, evidence, and counterexample checking.
1. System improvement is institutional version breeding—formal proposals, adversarial testing, shadow running, gray rollout, rollback guarantees—not ad-hoc updating.
1. Multi-system collaboration preserves sovereignty through formal treaties and isolation mechanisms rather than dissolving boundaries.
1. Long-term multi-system stability requires formal resource economics, capability licensing, precedent law, and diversity maintenance.

The path from "multiple models chatting" to "cognitive civilization" requires not more models or more parameters, but more governance, more compilation, more verification, and more institutional design. Whether this particular architecture is the right one remains to be proven empirically—and honestly, some of its later layers (V5-V6) may prove over-specified for the deployment scenarios that actually materialize. What seems clear is that the problem space it addresses—compilation, governance, verification, evolution, federation, economics—will need to be addressed by any system that aspires to reliable multi-agent coordination at scale.

A Complete Version Evolution Summary

Version	Period	Title	Core Contribution
V1	Early–Mid 2024	Protocol Backbone	Mother law compilation, 7 canonical pr
V2/V2.1	Late 2024–Early 2025	Runtime Civilization	Four execution loops, PromptOps, tool
V3	Mid 2025	Civilization-Level Evolution	Four domains, 12 formal planes, 5 close
V4	January–February 2026	Autonomous Self-Governance	5 hard problems, 5 new planes, meta-go
V5	March 2026	Federated Network	6 hard problems, 6 new planes, sovereig
V6	March–April 14, 2026	Constitutional Economy	6 problem classes, 6 new planes, balanc

Protocol Object	Introduced	Purpose
HPS	V1	Lossless human prompt preservation
ISG	V1	Intent decomposition and goal structuring
CRM	V1	Constraint and risk formalization
EBP	V1	Evidence collection and conflict detection
MIR	V1	Machine-readable intent representation
ECP	V1	Execution contract with action/validation gates
MHE	V1	High-trust inter-node handoff envelope
RouteDecision	V1	Value/risk/cost/uncertainty routing
TaskRuntimeState	V1	Runtime state tracking
PromptProgram	V2.1/V3	Compilable prompt strategy per node/task
ContextPackingPolicy	V2.1/V3	Context window orchestration rules
ConversationCheckpoint	V2.1/V3	Long-conversation memory hygiene
ModelProfile	V3	Model capability characterization
ToolCapabilityProfile	V3	Tool organ capability declaration
TeacherSessionRecord	V3	Teacher interaction audit trail
DistillationRecord	V3	Knowledge distillation tracking
PaperMechanismRecord	V3	Research mechanism extraction
UpgradeProposal	V3	Formal system upgrade proposal
CandidateVersion	V3	Version breeding candidate
AutonomyEnvelopePolicy	V4	Autonomy boundary definition
ResearchTransitPackage	V4	Research-to-production migration
PromotionContract	V4	Version promotion criteria
ControlPlaneSnapshot	V4	Civilization-level state observation
HeritageLedger	V4	Cross-version continuity management
CapabilityMap	V4	Self-model of strengths and weaknesses
SovereigntyProfile	V5	Civilization boundary definition
TreatyContract	V5	Cross-civilization formal agreement
FederationRouteDecision	V5	Multi-civilization task routing
TrustAttestation	V5	Cross-civilization output credibility
CivilizationHealthSnapshot	V5	Single-civilization health metrics
CivilizationForkRecord	V5	Formal fork event documentation
SanctionRecord	V5	Federation-level penalty tracking
FederationMemoryExchange	V5	Cross-civilization memory sharing
CivilizationalBalanceSheet	V6	Resource asset/liability accounting
CapabilityLicense	V6	Cross-civilization capability authorization
FederalTreasuryAllocation	V6	Federation resource distribution
PrecedentCaseRecord	V6	Jurisprudence accumulation
ExternalityReport	V6	System-level externality documentation
EcologyHealthSnapshot	V6	Federation diversity metrics
ScenarioPortfolio	V6	Macro strategic planning

B Protocol Object Inventory

C Comparative Analysis — ASA vs. Existing Systems

Dimension	AutoGen	CrewAI	MetaGPT	LangChain
Inter-agent communication	Natural language	Natural language	SOP-structured NL	Stateful NL
Compilation layer	None	None	SOP encoding	None
Constitutional governance	None	None	None	None
Formal verification	None	None	None	None
Routing intelligence	Round-robin / manual	Role-based delegation	Role assignment	Conditional
Rollback mechanism	None	Retry	None	Stateful
Evidence tracking	None	None	None	None
Ambiguity handling	Ignored	Ignored	Partial	Ignored
Teacher/frontier model role	Default worker	Default worker	Default worker	Default
Tool integration	Function calling	Tool use	Tool use	Tool use
Version evolution	Manual updates	Manual updates	Manual updates	Manual
Multi-system federation	Not addressed	Not addressed	Not addressed	Not
Economic model	None	None	None	None
Audit trail	Conversation log	Task log	Log files	Stateful
Scalability model	Horizontal (more agents)	Horizontal	Horizontal	Graph
Governance hierarchy	Flat	Manager/worker	Role hierarchy	Graph
Risk management	None	None	None	None
Research integration	None	None	None	None
Bio-inspired mechanisms	None	None	None	None
Formal protocol objects	0	0	0	0

C.0.1 Key Differentiators

1. **Compilation vs. Orchestration:** All existing frameworks orchestrate agents that consume natural language. ASA compiles natural language into machine protocols before execution begins—a categorical difference analogous to the gap between interpreted and compiled programming languages.
1. **Governance Depth:** Existing systems have at most one level of governance (manager/worker). ASA implements five governance levels: constitutional law → federation treaties → precedent law → route decisions → node contracts.
1. **Verification Rigor:** Existing systems rely on optional post-hoc review or simple output validation. ASA mandates a four-layer adversarial verification chain (structural, semantic, evidence, counterexample) that cannot be bypassed.
1. **Economic Equilibrium:** No existing system addresses the long-term economic sustainability of multi-agent collaboration. ASA’s V6 introduces formal resource economics, capability markets, externality accounting, and anti-monopoly mechanisms.

1. Evolutionary Continuity: Existing systems update through manual code changes. ASA’s version breeding system ensures that evolution is institutional—proposed, tested, shadowed, gray-released, and promoted under constitutional constraints with full lineage preservation.

References

- [1] Bai, Y., Kadavath, S., Kundu, S., et al. (2022). Constitutional AI: Harmlessness from AI Feedback. *arXiv:2212.08073*.
- [2] Chan, A., Salganik, M., et al. (2024). Governing AI Agents. *arXiv:2401.13138*.
- [3] Gao, D., Ji, H., et al. (2024). AgentScope: A Flexible yet Robust Multi-Agent Platform. *arXiv:2402.14034*.
- [4] Hammond, L., Hadfield, G. K., et al. (2025). Multi-Agent Risks from Advanced AI. *arXiv:2502.14143*.
- [5] Hong, S., Zhuge, M., et al. (2024). MetaGPT: Meta Programming for A Multi-Agent Collaborative Framework. *ICLR 2024*.
- [6] Koster, R., et al. (2022). Human-centred mechanism design with Democratic AI. *Nature Human Behaviour*, 6(10).
- [7] (2025). Decentralized Governance of AI Agents. *arXiv:2412.17114*.
- [8] McMahan, B., et al. (2017). Communication-Efficient Learning of Deep Networks from Decentralized Data. *AISTATS 2017*.
- [9] Mei, K., Li, Z., et al. (2024). AIOS: LLM Agent Operating System. *arXiv:2403.16971*.
- [10] Moura, J. (2024). CrewAI: Framework for Orchestrating Role-Playing Autonomous AI Agents. *GitHub/crewAI*.
- [11] Park, J. S., O’Brien, J. C., et al. (2023). Generative Agents: Interactive Simulacra of Human Behavior. *UIST 2023*.
- [12] Shavit, Y., et al. (2024). Practices for Governing Agentic AI Systems. *OpenAI Research*.
- [13] Wu, Q., Bansal, G., et al. (2024). AutoGen: Enabling Next-Gen LLM Applications via Multi-Agent Conversation. *ICLR 2024*.
- [14] Wu, Z., et al. (2024). OS-Copilot: Towards Generalist Computer Agents with Self-Improvement. *arXiv:2402.07456*.
- [15] Yang, Y., et al. (2025). AgentNet: Decentralized Evolutionary Coordination for LLM-based Multi-Agent Systems. *arXiv:2504.00587*.
- [16] Dennis, L., Fisher, M., et al. (2024). Verifiable Autonomy and Rational Agents: From Theory to Practice. *Springer LNAI*, vol. 14462.

- [17] (2025). A Trace-Based Assurance Framework for Agentic AI Orchestration. *arXiv:2603.18096*.
- [18] Anthropic. (2023). Collective Constitutional AI: Aligning a Language Model with Public Input. *arXiv:2310.12672*.
- [19] Wang, L., Ma, C., et al. (2024). A Survey on Large Language Model based Autonomous Agents. *Frontiers of Computer Science*, 18(6).
- [20] Talebirad, Y., Nadiri, A. (2024). Multi-Agent Collaboration: Harnessing the Power of Intelligent LLM Agents. *arXiv:2306.03314*.
- [21] Khattab, O., et al. (2024). DSPy: Compiling Declarative Language Model Calls into Self-Improving Pipelines. *ICLR 2024*.
- [22] Sumers, T. R., et al. (2024). Cognitive Architectures for Language Agents. *TMLR 2024*.
- [23] Anthropic. (2024). Model Context Protocol (MCP): An Open Standard for AI-Tool Integration. *Technical Report*.
- [24] Google. (2025). Agent-to-Agent (A2A) Protocol: Open Interoperability for AI Agents. *Technical Specification*.
- [25] Zhuge, M., et al. (2024). GPTSwarm: Language Agents as Optimizable Graphs. *ICML 2024*.
- [26] Chen, W., et al. (2024). AgentVerse: Facilitating Multi-Agent Collaboration and Exploring Emergent Behaviors. *ICLR 2024*.